

Làm việc với nhóm dữ liệu với Group By và Having

Trong nội dung học trước, chúng ta đã chuyển đổi vai trò từ việc liệt kê chi tiết bản ghi sang thống kê dữ liệu thông qua các hàm tổng hợp như **COUNT**, **SUM**, **AVG**. Chúng ta cũng đã làm quen với mệnh đề **GROUP BY** – công cụ dùng để phân loại dữ liệu vào các nhóm (tương tự như việc chia các bản ghi vào từng giỏ dữ liệu riêng biệt).

Ví dụ, thay vì tính điểm trung bình của toàn trường bằng lệnh **SELECT AVG(DiemTrungBinh) FROM SinhVien;**, chúng ta đã có thể phân nhóm theo lớp:

```
SELECT lop_hoc_id, AVG(DiemTrungBinh)
FROM SinhVien
GROUP BY lop_hoc_id;
```

Truy vấn này trả về danh sách các lớp kèm theo điểm trung bình tương ứng của từng lớp đó. Tuy nhiên, một vấn đề phát sinh khi chúng ta cần lọc dữ liệu dựa trên kết quả của các hàm tổng hợp này.



1. Phân biệt đối tượng lọc dữ liệu

Giả sử sau khi có danh sách điểm trung bình của 100 lớp học, yêu cầu đặt ra là: "Chỉ hiển thị những lớp học có điểm trung bình trên 8.0".

Lỗi phổ biến của người mới bắt đầu là sử dụng mệnh đề **WHERE** để giải quyết yêu cầu này:

```
-- Câu lệnh sai logic
SELECT lop_hoc_id, AVG(DiemTrungBinh)
FROM SinhVien
WHERE AVG(DiemTrungBinh) > 8.0
GROUP BY lop_hoc_id;
```

Tại sao mệnh đề **WHERE** lại không hợp lệ trong trường hợp này?

Để hiểu rõ, chúng ta cần xem xét thứ tự thực thi của một câu lệnh SQL:

1. **FROM**: Xác định nguồn dữ liệu.
2. **WHERE**: Kiểm tra và lọc từng bản ghi thô (từng hàng dữ liệu đơn lẻ).
3. **GROUP BY**: Gom nhóm các bản ghi đã vượt qua bộ lọc **WHERE**.
4. **SELECT (Aggregate Functions)**: Thực hiện tính toán tổng hợp (như **AVG**) trên từng nhóm.

Tại bước số 2, khi mệnh đề **WHERE** đang thực thi, hệ quản trị cơ sở dữ liệu đang xem xét các cá nhân sinh viên và chưa thực hiện việc phân nhóm hay tính toán trung bình. Do đó, hệ thống không thể biết giá trị **AVG(DiemTrungBinh)** của một nhóm là bao nhiêu để thực hiện so sánh. Mệnh đề **WHERE** chỉ có thể lọc dữ liệu thô, không thể lọc dữ liệu đã qua tổng hợp.

2. Giải pháp: Mệnh đề HAVING

Để lọc các nhóm sau khi đã thực hiện tính toán tổng hợp, chúng ta sử dụng mệnh đề **HAVING**. Mệnh đề này đóng vai trò là bộ lọc hoạt động dựa trên các kết quả của hàm tổng hợp sau khi dữ liệu đã được nhóm lại bằng **GROUP BY**.

Cú pháp chuẩn:

```
SELECT lop_hoc_id, AVG(DiemTrungBinh)
FROM SinhVien
GROUP BY lop_hoc_id
HAVING AVG(DiemTrungBinh) > 8.0;
```

Hệ thống sẽ thực hiện theo quy trình:

- Chia tất cả sinh viên vào các nhóm dựa trên `lop_hoc_id`.
- Tính toán điểm trung bình cho từng nhóm.
- Mệnh đề `HAVING` sẽ kiểm tra giá trị trung bình vừa tính và loại bỏ các nhóm không đạt tiêu chuẩn (nhỏ hơn hoặc bằng 8.0).

3. So sánh WHERE và HAVING

Việc phân biệt rõ ràng hai mệnh đề này là yêu cầu bắt buộc đối với một chuyên viên quản trị cơ sở dữ liệu.

Đặc điểm so sánh	Mệnh đề WHERE	Mệnh đề HAVING
Đối tượng tác động	Tác động lên từng dòng dữ liệu thô.	Tác động lên các nhóm dữ liệu đã tổng hợp.
Vị trí thực thi	Đứng trước <code>GROUP BY</code> .	Đứng sau <code>GROUP BY</code> .
Hàm tổng hợp	Không được dùng kèm hàm tổng hợp (COUNT, SUM...).	Được dùng kèm với các hàm tổng hợp.

4. Kết hợp WHERE và HAVING trong truy vấn phức tạp

Trong thực tế, một câu lệnh thường kết hợp cả hai mệnh đề lọc để giải quyết các bài toán đa tầng.

Ví dụ: Tính điểm trung bình theo lớp, nhưng chỉ xét những sinh viên cư trú tại Hà Nội và sau khi tính xong, chỉ hiển thị những lớp có trên 10 sinh viên thỏa mãn điều kiện đó.



```
SELECT lop_hoc_id, AVG(DiemTrungBinh)
FROM SinhVien
WHERE QueQuan = 'Hà Nội'           -- Lọc bản ghi thô trước khi nhóm
GROUP BY lop_hoc_id               -- Phân nhóm theo lớp
HAVING COUNT(id) > 10;            -- Lọc nhóm dựa trên số lượng thành viên
```

Tổng kết

Bài học đã hoàn thiện kỹ năng lập báo cáo chuyên sâu thông qua việc kết hợp các mệnh đề:

1. **GROUP BY**: Phân loại dữ liệu thành các nhóm đối tượng.
2. **HAVING**: Thiết lập bộ lọc cho các nhóm dựa trên kết quả tính toán tổng hợp.

Thứ tự chuẩn xác của một câu truy vấn SQL đầy đủ là:

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT.

Việc nắm vững thứ tự này giúp lập trình viên kiểm soát chính xác luồng dữ liệu, từ khâu lọc dữ liệu đầu vào đến khâu tổng hợp và trình bày báo cáo đầu ra.